

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# **01**)

Apply modern simulation tools Model Sim and Logisim to implement basic processor components

Student Name: __Areeb Fatima__ Roll No: ____24I-6543____ Section: _B_

Date performed: ____19th August____, 2026

MARKS AWARDED: ____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task, we were asked to prove the basic logic gates wave through Verilog codes of Gate Level Abstraction. For this, we wrote the basic code infrastructure and testbench which is common for all the gates except for not gate as only one input and output is there in not gate and in code for proving all the basic gates, we just change the keywords in code and got the simulated wave for all the gates.

Code:**And Gate:**

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
and a1(c,a,b);  
  
endmodule
```

Or Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
or a1(c,a,b);  
  
endmodule
```

Not Gate:

```
module LogicGates(a,b);  
  
input a;  
  
output b;
```

```
not a1(b,a);

endmodule

module testbench();

reg x;

wire z;

LogicGates uut(x,z);

// LogicGates uut(.a(x),.b(z));

initial

begin

x=0;

#50

x=1;

end

endmodule
```

Nand Gate:

```
module LogicGates(a,b,c);

input a;

input b;

output c;

nand a1(c,a,b);

endmodule
```

Nor Gate:

```
module LogicGates(a,b,c);
```

```
input a;  
  
input b;  
  
output c;  
  
nor a1(c,a,b);  
  
endmodule
```

Xor Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
xor a1(c,a,b);  
  
endmodule
```

Xnor Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
xnor a1(c,a,b);  
  
endmodule
```

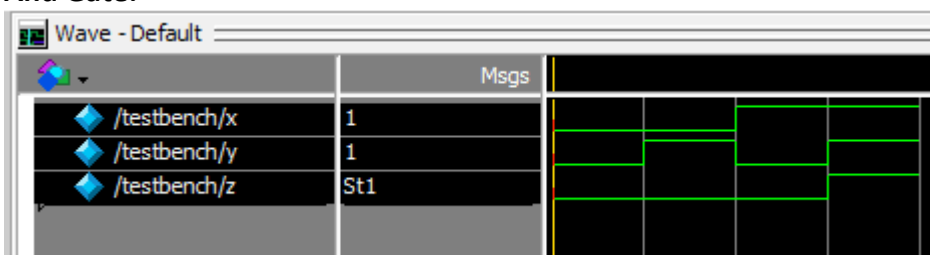
Common testbench:

```
module testbench();  
  
reg x;  
  
reg y;
```

```
wire z;  
  
LogicGates uut(x,y,z);  
  
// LogicGates uut(.a(x),.b(y),.c(z));  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50  
  
x=0; y=1;  
  
#50  
  
x=1; y=0;  
  
#50  
  
x=1; y=1;  
  
end  
  
endmodule
```

Output:

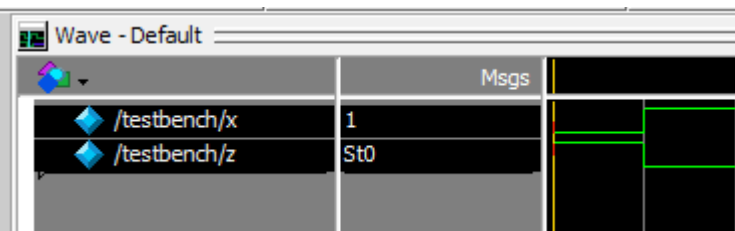
And Gate:



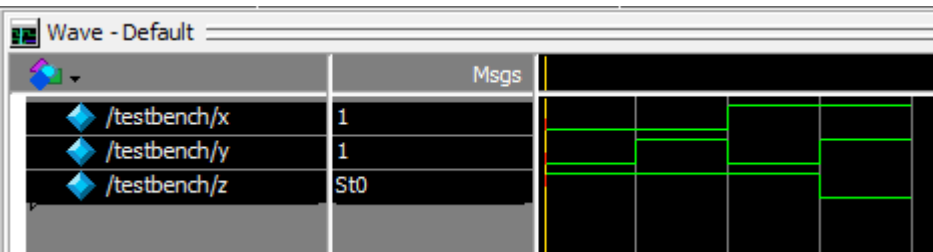
Or Gate:



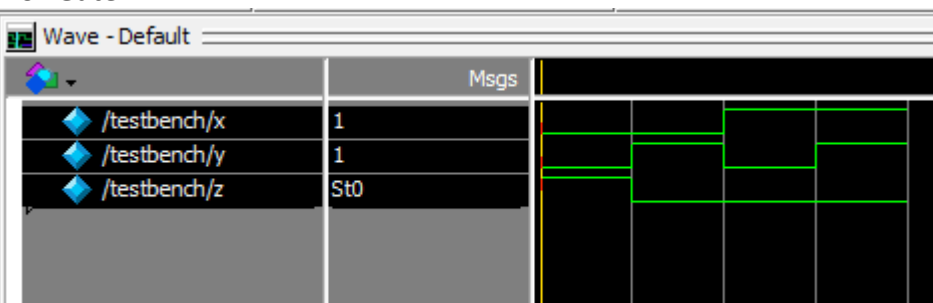
Not Gate:



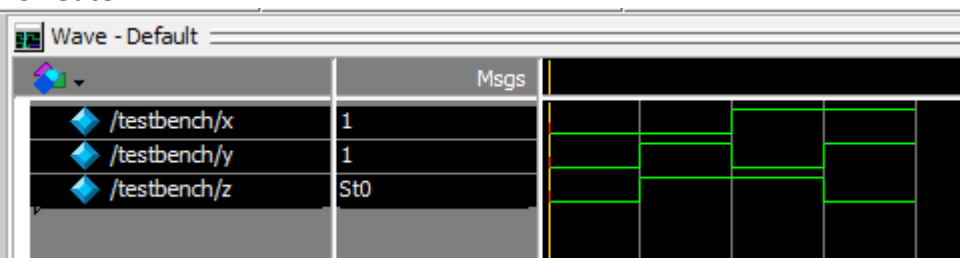
Nand Gate:



Nor Gate:



Xor Gate:



Xnor Gate:

Wave - Default		Msgs			
	/testbench/x	1			
	/testbench/y	1			
	/testbench/z	St1			

Task 2:

Description:

In this task, we were asked to prove the basic logic gates wave form through Verilog codes in Data Flow modeling. For this, we wrote the basic code infrastructure and testbench which is common for all the gates except for not gate as only one input and output is there in not gate and in code for proving all the basic gates, we just change the symbols for the particular gates in code and got the simulated wave for all the gates.

Code:

And Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
assign c = a & b;  
  
endmodule
```

Or Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
assign c = a | b;  
  
endmodule
```

Not Gate:

```
module LogicGates(a,b);  
  
input a;  
  
output b;
```

```
assign b = ~a;

endmodule

module testbench();

reg x;

wire z;

LogicGates uut(x,z);

initial

begin

x=0;

#50

x=1;

end

endmodule
```

Nand Gate:

```
module LogicGates(a,b,c);

input a;

input b;

output c;

assign c = ~(a & b);

endmodule
```

Nor Gate:

```
module LogicGates(a,b,c);

input a;
```

```
input b;  
  
output c;  
  
assign c = ~(a | b);  
  
endmodule
```

Xor Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
assign c = a ^ b;  
  
endmodule
```

Xnor Gate:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
assign c = ~(a ^ b);  
  
endmodule
```

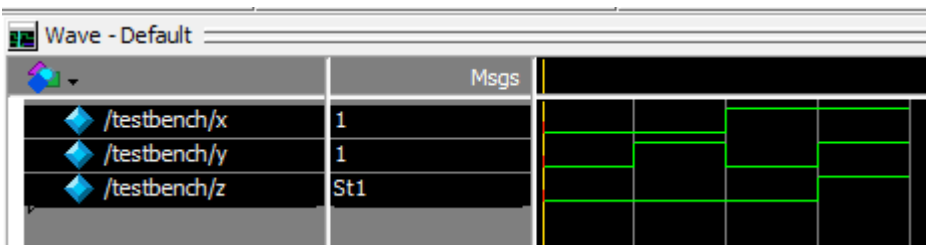
Common Testbench (Except Not Gate):

```
module testbench();  
  
reg x;  
  
reg y;  
  
wire z;  
  
LogicGates uut(x,y, z);
```

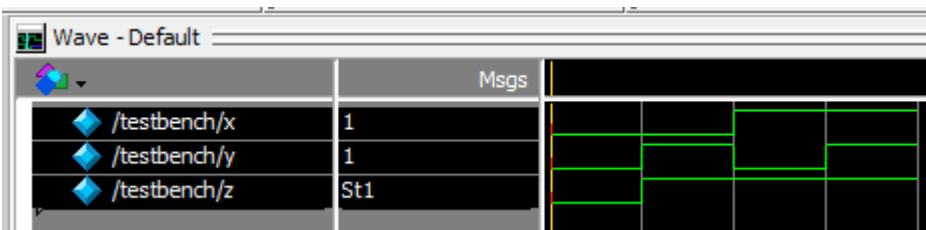
```
initial  
  
begin  
  
x=0; y=0;  
  
#50  
  
x=0; y=1;  
  
#50  
  
x=1; y=0;  
  
#50  
  
x=1; y=1;  
  
end  
  
endmodule
```

Output:

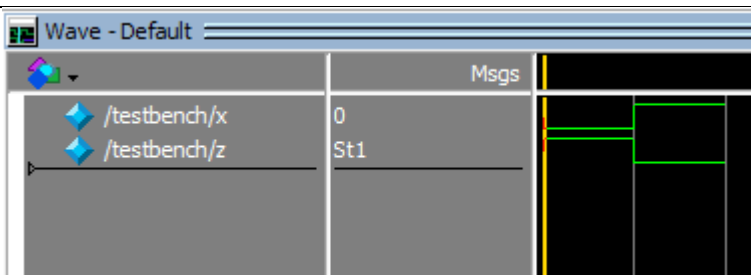
And Gate:



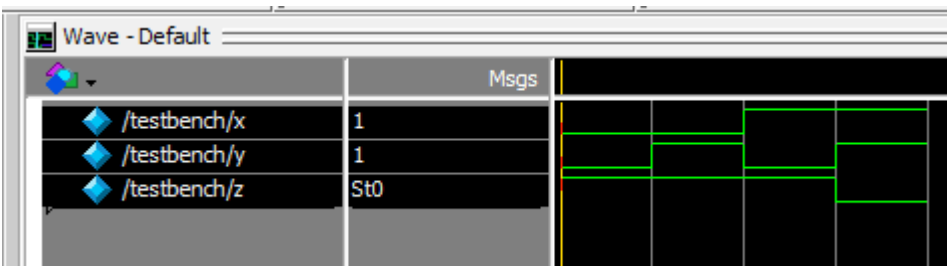
Or Gate:



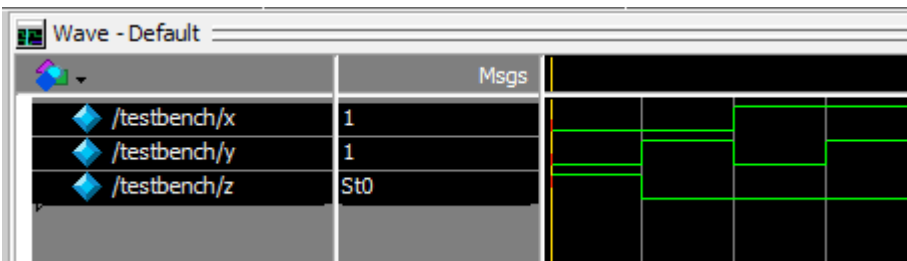
Not Gate:



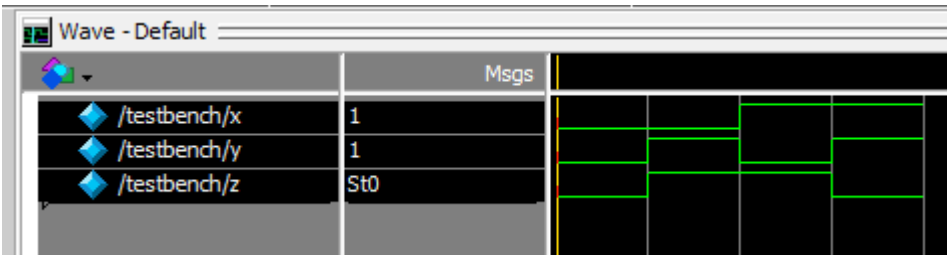
Nand Gate:



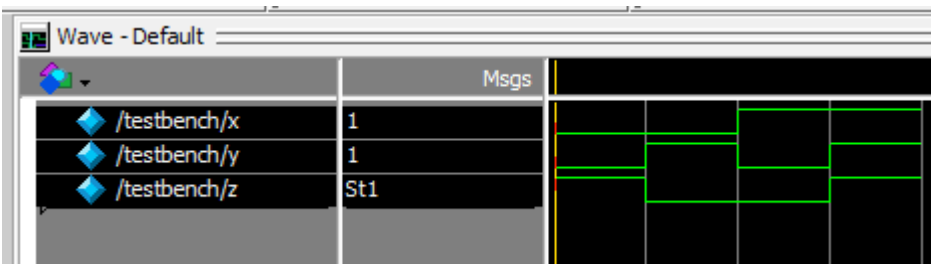
Nor Gate:



Xor Gate:



Xnor Gate:

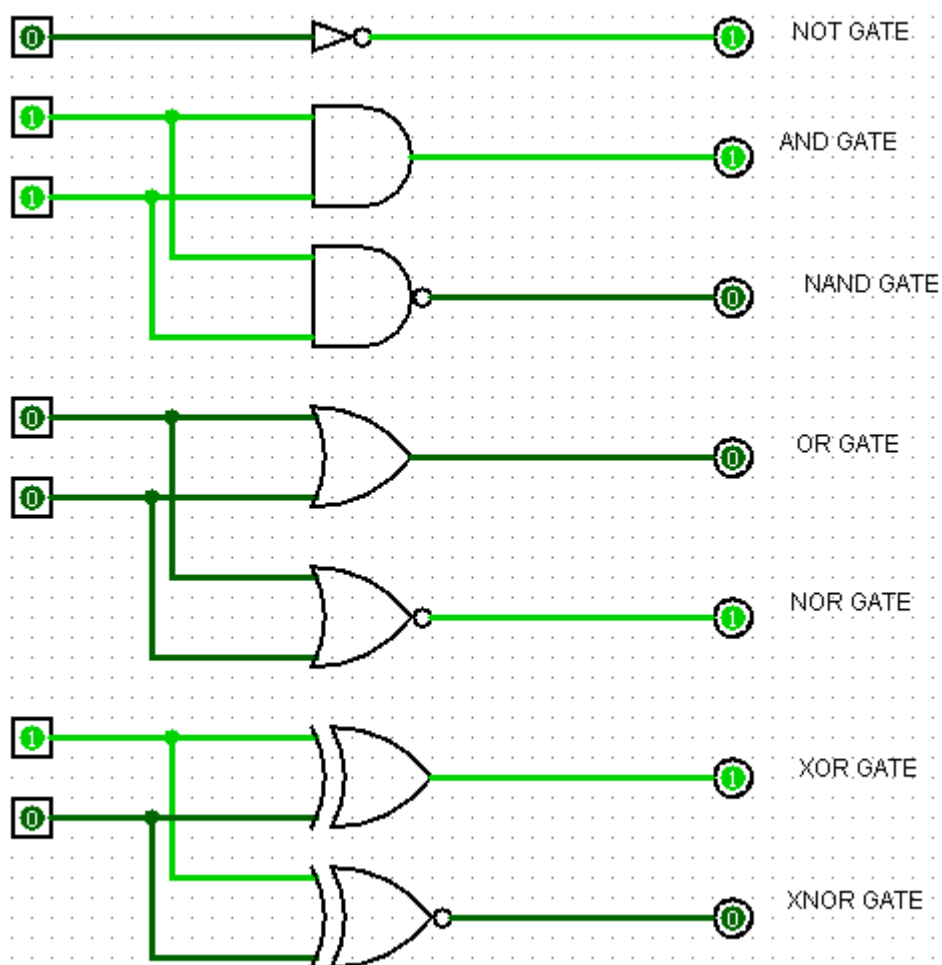


Task 3:

Description:

In this task, we were asked to test the working of Basic Logic Gates through Logisim. So, first we install the Logisim software and in workspace we simply extract the gates and gave input and output to the circuit and label them and attached the circuit input and output through connecting wires. For more visible understanding I connect AND with NAND, OR with NOR and XOR with XNOR to understand their alter behavior for circuit.

Output:



Task 4:

Description:

In this task, and we were asked to write the Verilog code of circuit using both Gate Level and Data Flow modeling. So, for that we created the wires which gave the output of midway circuit, as a (NOT) gate

compliment of a so, we wrote the output as t1 and b NOT is t2 and then t1(a NOT) ANDED with b as given in circuit and t2(b NOT) ANDED with a and then in last t3 is OR with t4 to give the required output which is the circuit of basically **a XOR b** and in this way, we prove the wave through both modeling.

Code:

Gate Level Modeling:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
wire t1;  
  
wire t2;  
  
wire t3;  
  
wire t4;  
  
output c;  
  
not n1(t1,a);  
  
and a1(t3,b,t1);  
  
not n2(t2,b);  
  
and a2(t4,t2,a);  
  
or o1(c,t3,t4);  
  
endmodule;
```

Data Flow Modeling:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
wire t1;  
  
wire t2;
```

```
wire t3;  
  
wire t4;  
  
output c;  
  
assign t1= ~a;  
  
assign t3= t1 & b;  
  
assign t2= ~b;  
  
assign t4= a & t2;  
  
assign c= t3 | t4;  
  
endmodule;
```

Common testbench:

```
module testbench();  
  
reg x;  
  
reg y;  
  
wire z;  
  
LogicGates uut(x,y,z); //unit under test  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50  
  
x=0; y=1;  
  
#50  
  
x=1; y=0;  
  
#50
```

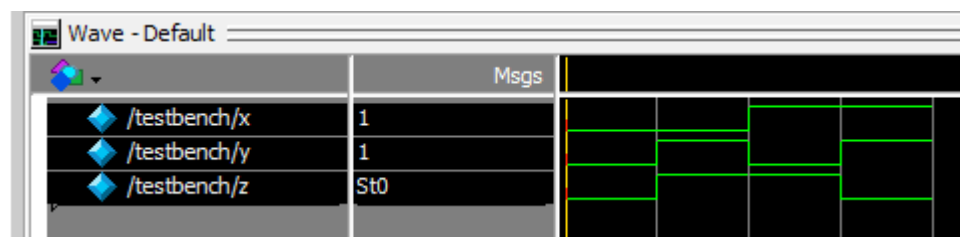
```
x=1; y=1;

end

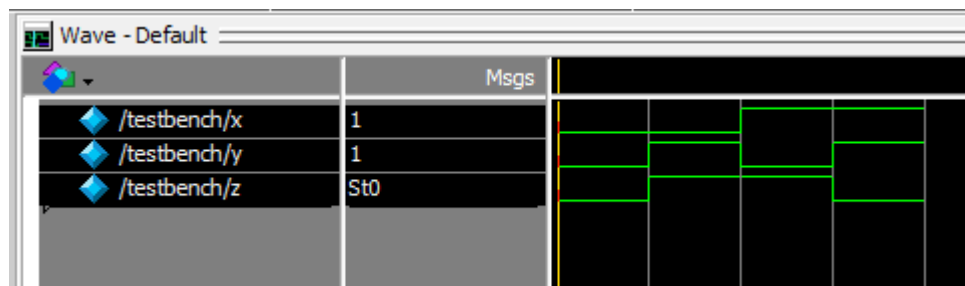
endmodule;
```

Output:

Gate Level Modeling:



Data Flow Modeling:

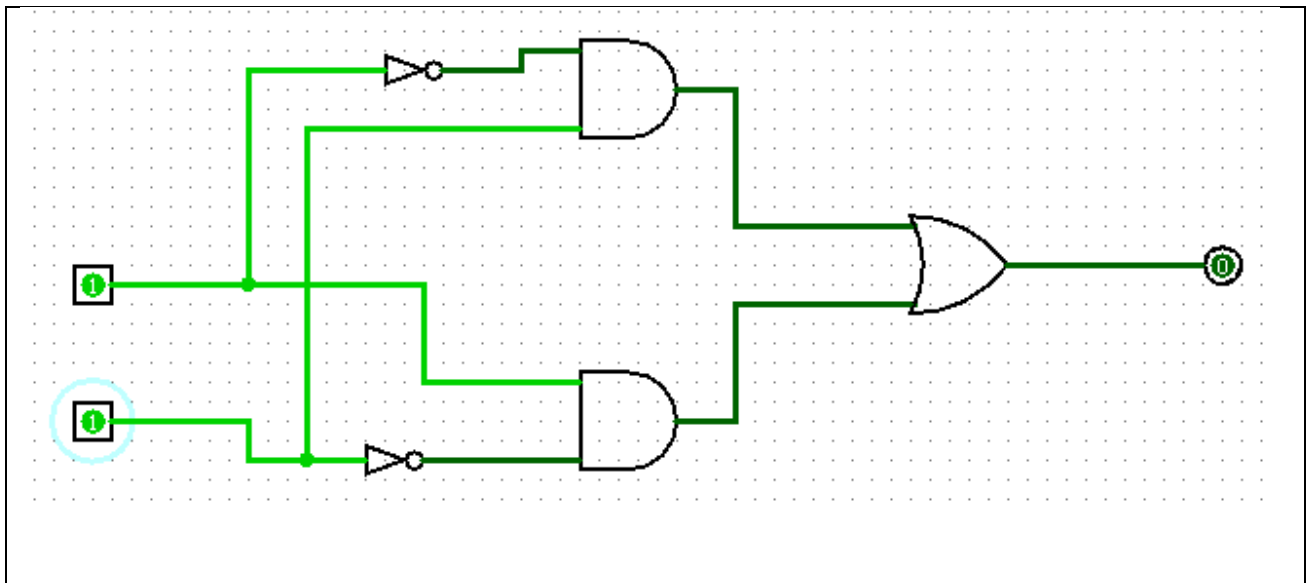


Task 5:

Description:

In this task, we were asked to test the working of above circuit which is the circuit of XOR gate through Logisim. So, we created a workspace and extract the basic gates as AND, OR and NOT and gave the wiring to the circuit as required to give the accurate output, we also attached the input and output symbols to the circuit through wiring and verify the XOR gate by giving different inputs.

Output:



Task 6:

Description:

In this task, we were asked to find the 1's complement of four bits as 1's complement is the NOT of the bits provided so we just wrote the code by taking 4 inputs and 4 outputs and then we took not for Gate level modeling of four bits and tilde symbol for Data Flow modeling and in testbench which is common for both modeling we make 16 different combinations for 4 bit number and verify the results for all the combinations through wave form output.

Code:

Gate Level Modeling:

```
module complement(a1,a2,a3,a4,b1,b2,b3,b4);

input a1,a2,a3,a4;

output b1,b2,b3,b4;

not n1(b1,a1);

not n2(b2,a2);

not n3(b3,a3);

not n4(b4,a4);

endmodule;
```

Data Flow Modeling:

```
module complement(a1,a2,a3,a4,b1,b2,b3,b4);
```

```
input a1,a2,a3,a4;
```

```
output b1,b2,b3,b4;
```

```
assign b1= ~a1;
```

```
assign b2= ~a2;
```

```
assign b3= ~a3;
```

```
assign b4= ~a4;
```

```
endmodule;
```

Common testbench:

```
module testbench();
```

```
reg a,b,c,d;
```

```
wire w,x,y,z;
```

```
complement uut(a,b,c,d,w,x,y,z);
```

```
initial
```

```
begin
```

```
a=0; b=0; c=0; d=0;
```

```
#50
```

```
a=0; b=0; c=0; d=1;
```

```
#50
```

```
a=0; b=0; c=1; d=0;
```

```
#50
```

```
a=0; b=0; c=1; d=1;
```

```
#50
```

a=0; b=1; c=0; d=0;

#50

a=0; b=1; c=0; d=1;

#50

a=0; b=1; c=1; d=0;

#50

a=0; b=1; c=1; d=1;

#50

a=1; b=0; c=0; d=0;

#50

a=1; b=0; c=0; d=1;

#50

a=1; b=0; c=1; d=0;

#50

a=1; b=0; c=1; d=1;

#50

a=1; b=1; c=0; d=0;

#50

a=1; b=1; c=0; d=1;

#50

a=1; b=1; c=1; d=0;

#50

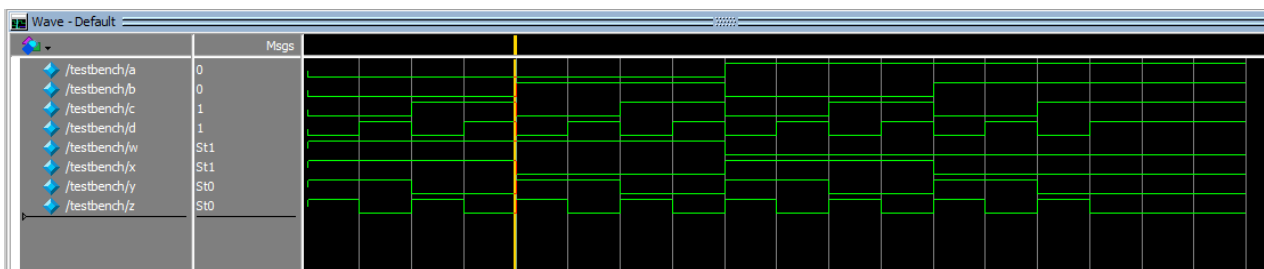
a=1; b=1; c=1; d=1;

end

endmodule;

Output:

Gate Level Modeling:



Data Flow Modeling:

